INVESTIGATING DEEP Q-LEARNING MODIFICATIONS WITH RANDOMISED CONTROL TRIALS

Matimu Khosa
School of Computer Science and Applied Mathematics
University of the Witwatersrand
Johannesburg, South Africa
2716924@students.wits.ac.za

Dr. Steven James (Supervisor)
School of Computer Science and Applied Mathematics
University of the Witwatersrand
Johannesburg, South Africa

Prof. Benjamin Rosman (Supervisor)
School of Computer Science and Applied Mathematics
University of the Witwatersrand
Johannesburg, South Africa

***Abstract*—Extensions to the deep Q-network (DQN) are usually evaluated with their own training recipe, their own tuned hyperparameters and a handful of random seeds, so a reported gain may belong to the setting rather than to the extension. We test four extensions, namely Double DQN, prioritised experience replay (PER), multi-step returns and Averaged DQN, in a paired randomised control trial on the MinAtar suite. Thirty hyperparameter configurations were sampled from a shared population, and every configuration was trained with DQN and with each extension, so that each extension is compared with DQN on exactly the same configuration and seed. The outcome is the area under the evaluation curve, a measure of sample efficiency. Our central finding is that the apparent benefit of an extension depends strongly on the training recipe and the hyperparameters it is tested with. Under one shared recipe, in a population whose learning rates mostly lie above the best rate for DQN, Averaged DQN improved on DQN in three of four games, Double DQN helped little, and PER and multi-step returns reduced performance, unlike in earlier component studies that used different recipes. Smaller, exploratory experiments suggest why. With the recipe and configuration of an earlier MinAtar study, our PER and multi-step agents matched or beat DQN, as that study reported, and lowering the learning rate made DQN much stronger in most games and removed most of Averaged DQN's advantage. These experiments point to an interaction between the optimiser, the learning rate and the extension, although they do not isolate its cause. Evaluations based on one checkpoint, a few configurations or the best configuration of each method would have missed or even reversed some of these effects. Conclusions about which DQN extensions matter should therefore be read as conditional on the recipe and configurations under which they were obtained.**

***Index Terms*—deep reinforcement learning, deep Q-network, experimental design, randomised control trial, MinAtar, sample efficiency**

I. Introduction

The deep Q-network (DQN) of Mnih et al. [2015] learnt to play Atari 2600 games from raw pixels. It did so by combining Q-learning with a neural network and two stabilising devices: a replay buffer of past experience and a target network that is updated only periodically. DQN became the base on which many later methods were built.

Many extensions followed, and each changed one part of DQN. Double DQN changes how the learning target is computed [van Hasselt et al. 2016], prioritised experience replay (PER) changes which past experiences are replayed [Schaul et al. 2016], multi-step (n-step) returns look further ahead before bootstrapping [Hernandez-Garcia and Sutton 2019; Sutton and Barto 2018], and Averaged DQN averages several earlier target networks [Anschel et al. 2017]. Rainbow combined several such components and found prioritised replay and multi-step returns to be the most important of them [Hessel et al. 2018].

Each extension was, however, introduced with its own training recipe and evaluated on its own tuned configuration. Deep reinforcement learning is sensitive to hyperparameters, random seeds and implementation details [Henderson et al. 2018], and comparisons often rest on a few runs [Agarwal et al. 2021]. When an extension beats DQN under one tuned configuration, we therefore cannot tell whether the gain belongs to the extension or to the configuration.

We address this problem with a randomised control trial, following Pretorius et al. [2020]. In a medical trial, patients are drawn from a population and receive treatments; here, the "patients" are hyperparameter configurations drawn at random from a shared population, and the "treatments" are the extensions. Every configuration receives every treatment, so each extension is compared with DQN on exactly the same configuration and seed. This pairing removes the effect of the configuration from each comparison, while the random sampling lets the results speak for the population rather than for one hand-picked setting. We use the MinAtar suite [Young and Tian 2019], whose small games make hundreds of full training runs affordable on ordinary CPUs.

We ask three research questions.

- *RQ1:* under one shared training recipe and across a population of configurations, does each extension change sample efficiency relative to DQN?
- *RQ2:* where our results differ from published ones, why? In particular, do the effects depend on the training recipe

and on the learning rate?
- *RQ3:* would smaller or more typical evaluations, based on one checkpoint, a few configurations or the best configuration of each method, have reached the same conclusions?

Our central contribution is not a ranking of the extensions. It is evidence that the conclusion one reaches about an extension depends strongly on the training recipe and the hyperparameter range it is evaluated with, and that common evaluation shortcuts can hide or reverse this dependence. Specifically, we contribute the following.

- A paired trial of four DQN extensions on 30 sampled configurations and four MinAtar games, 600 runs in total, in which, under the shared recipe, only Averaged DQN improves on DQN in more than one game, while PER and n-step returns reduce performance.
- A positive control in which the same PER and n-step implementations help with the recipe and configuration of an earlier MinAtar study, which suggests that their deficit in the main trial is specific to our recipe rather than to the extensions or to our code.
- Evidence that the learning rate matters as much as any extension, and exploratory evidence that the advantage of Averaged DQN largely disappears at smaller learning rates, which points to an interaction between the learning rate and the extension.
- Evidence that single-checkpoint, small and best-configuration evaluations can miss or reverse these effects, and practical recommendations for evaluating reinforcement learning algorithms that follow from our results.

Section II explains DQN and the four extensions, Section III describes the trial and the additional experiments, Section IV reports the results and Section V describes related work. Section VI separates what the main trial establishes from what the additional experiments suggest, relates the findings to earlier component studies and draws practical implications. Section VII states the limitations and Section VIII concludes.

II. Background

This section builds up the algorithms we compare, starting from the reinforcement learning problem and ending with the four extensions. Each equation is followed by what it means in words.

A. Reinforcement Learning

In reinforcement learning, an agent learns by trial and error while interacting with an environment (Fig. 1). At each time step t, the agent observes a state s_t, chooses an action a_t, receives a reward r_{t+1} and moves to a new state s_{t+1}. This interaction is formalised as a Markov decision process, in which the next state and reward depend only on the current state and action [Sutton and Barto 2018].

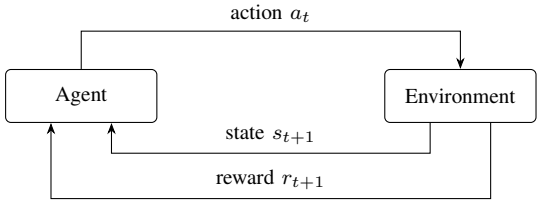


Fig. 1. The agent–environment interaction in a Markov decision process, after Fig. 3.1 of Sutton and Barto [2018].

The agent's goal is to collect as much reward as possible over time. It therefore maximises the *return*, the discounted sum of future rewards,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (1)$$

The discount factor $\gamma \in [0, 1)$ makes a reward received k steps in the future worth γ^k of the same reward received now. With $\gamma = 0.99$, as in this work, a reward 100 steps away still counts for about a third of its value, so the agent plans far ahead.

To choose actions, the agent estimates how good each action is. The action-value function $Q^\pi(s, a)$ is the expected return when the agent takes action a in state s and then follows its policy π. The best possible action values, $Q^*(s, a) = \max_\pi Q^\pi(s, a)$, satisfy the Bellman optimality equation [Sutton and Barto 2018],

$$Q^*(s, a) = \mathbb{E}\Big[r + \gamma \max_{a'} Q^*(s', a')\Big], \quad (2)$$

where s' is the next state and the expectation is over the reward and next state. In words, the value of an action is the reward it brings plus the discounted value of the best action available afterwards. Once Q^* is known, acting optimally is simple: in each state, choose the action with the largest value.

B. Q-Learning and the Deep Q-Network

Q-learning [Watkins and Dayan 1992] learns Q^* from experience by turning (2) into an update. After observing a transition (s, a, r, s'), it computes the *temporal-difference (TD) error*

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a), \quad (3)$$

the gap between a one-step estimate of the value, $r + \gamma \max_{a'} Q(s', a')$, and the current estimate $Q(s, a)$. It then moves $Q(s, a)$ a small step in the direction of δ. Using the agent's own estimate of the next state's value in place of the true return is called *bootstrapping*.

DQN [Mnih et al. 2015] represents Q with a neural network $Q(s, a; \theta)$ with weights θ, so that it can handle states such as images, which are too many to list in a table. Three ideas make this work. First, transitions are stored in a replay buffer $\mathcal{D}$ [Lin 1992], and the network learns from random mini-batches drawn from it; this breaks the strong correlation between consecutive transitions and lets each transition be

reused. Second, the network is trained to minimise the squared TD error,

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim U(\mathcal{D})}\Big[\big(y - Q(s,a;\theta)\big)^2\Big], \tag{4}$$

where $U(\mathcal{D})$ means that transitions are drawn uniformly from the buffer. Third, the learning target

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-) \tag{5}$$

is computed with a separate *target network* whose weights θ^- are a copy of θ that is refreshed only every C steps. Without it, every update would also move the target, and the network would chase its own tail. When s' ends the episode, there is no future value and the target is simply $y = r$.

The agent explores with an ε-greedy policy: with probability ε it takes a random action, and otherwise the action with the largest estimated value. Here ε starts at 1, so the agent first acts entirely at random, and is lowered linearly to a small final value.

C. The Four Extensions

Each extension below changes exactly one part of DQN: the target, the sampling of the replay buffer, the horizon of the target, or the target network.

Double DQN [van Hasselt et al. 2016] reduces overestimation. The maximum in (5) uses the same noisy estimates both to pick the best next action and to value it. When the estimates contain random errors, the action that looks best is often one whose value was overestimated, so the maximum of the estimates tends to exceed the true maximum, and the error is passed on through bootstrapping. Double DQN separates the two roles: the online network θ picks the action and the target network θ^- values it,

$$y = r + \gamma\, Q\big(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-\big). \tag{6}$$

Because the two networks' errors are partly independent, an action that is overestimated by one is less likely to be overestimated by the other.

Prioritised experience replay [Schaul et al. 2016] replays surprising transitions more often. Each transition i has a priority $p_i = |\delta_i| + \kappa$, its latest absolute TD error plus a small constant κ that keeps every transition sampleable. Transition i is drawn with probability

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}. \tag{7}$$

The exponent α sets how strongly the priorities count: $\alpha = 0$ gives uniform sampling, as in DQN, and $\alpha = 1$ samples in direct proportion to the TD error. New transitions receive the largest priority seen so far, so that each is replayed at least once.

Sampling by priority changes the data the network learns from, and therefore the expected gradient. PER corrects this with importance-sampling weights,

$$w_i = \frac{\big(|\mathcal{D}|\, P(i)\big)^{-\beta}}{\max_{j\in\mathcal{D}} \big(|\mathcal{D}|\, P(j)\big)^{-\beta}}, \tag{8}$$

where $|\mathcal{D}|$ is the number of transitions in the buffer, and the maximum in the denominator is attained by the transition with the lowest priority. Uniform sampling would draw transition i with probability $1/|\mathcal{D}|$; when $\beta = 1$, multiplying its loss by the numerator alone, $(|\mathcal{D}|\, P(i))^{-1}$, exactly undoes the over- or under-sampling in expectation. The denominator divides every weight by the same number, so that every weight is at most one; this rescales the expected gradient without changing its direction, so after normalisation the correction is exact only up to this common factor, which acts like a smaller step size. PER starts with a partial correction, $\beta = \beta_0 < 1$, and raises β linearly to 1 by the end of training, when an unbiased gradient matters most [Schaul et al. 2016]. Each transition's loss in (4) is multiplied by its weight w_i.

Multi-step returns [Hernandez-Garcia and Sutton 2019; Sutton and Barto 2018] look n steps ahead before bootstrapping:

$$y = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n \max_{a'} Q(s_{t+n}, a'; \theta^-). \tag{9}$$

The first term is the discounted reward actually observed over the next n steps; the second is the estimated value of the state reached after them. A reward therefore reaches the value of a state n steps earlier in a single update, instead of creeping back one step per update. The price is bias: the n observed rewards come from the exploring behaviour policy, not the greedy policy whose value is being learnt. Like Rainbow [Hessel et al. 2018], we apply no off-policy correction, so this difference is ignored.

Averaged DQN [Anschel et al. 2017] averages the last K target networks $\theta_1^-, \ldots, \theta_K^-$ before taking the maximum:

$$y = r + \gamma \max_{a'} \frac{1}{K} \sum_{k=1}^{K} Q(s', a'; \theta_k^-). \tag{10}$$

Each target network makes its own approximation error. Averaging K estimates reduces the variance of their error, by a factor of K if the errors were uncorrelated and by less when they are correlated, which gives a steadier target [Anschel et al. 2017]. The cost is K forward passes per target; the number of gradient updates is the same as in DQN.

III. Method

This section first describes the main trial: its design, the environments, the training recipe, the outcome and the statistics. It then describes three additional experiments: two designed to help explain the results, and one that repairs the evaluation of Seaquest.

A. Trial Design

Fig. 2 summarises the design. The population Λ contains every combination of the candidate values of six shared hyperparameters (Table I). Each of the $N = 30$ individuals λ_j was drawn by sampling each hyperparameter independently and uniformly from its candidates with a seeded random generator, and was given its own training seed, $1337 + j$.

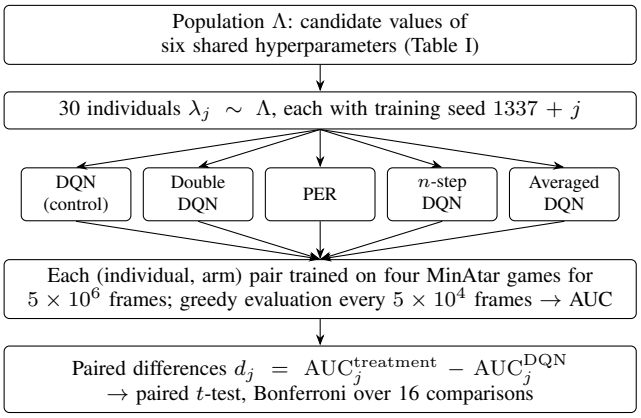


Fig. 2. Design of the main trial. Every individual (a hyperparameter configuration with its own seed) is trained under all five arms, so each treatment is compared with DQN on the same configuration.

TABLE I
SHARED HYPERPARAMETER POPULATION AND THE SAMPLED INDIVIDUALS

Hyperparameter	Values (individuals in sample)
Learning rate	10^{-4} (7), $5{\times}10^{-4}$ (13), 10^{-3} (10)
Replay buffer capacity	10^4 (15), $5{\times}10^4$ (5), 10^5 (10)
Mini-batch size	32 (15), 64 (15)
Target update period (steps)	500 (8), 1000 (11), 2000 (11)
ε-decay duration (steps)	$5{\times}10^4$ (11), 10^5 (7), $2{\times}10^5$ (12)
Hidden layers	1 (16), 2 (14)

Because this is a simple random sample, the counts in Table I are not perfectly balanced.

The population defines the question the trial answers: the effect of an extension averaged over configurations that a practitioner might reasonably use with DQN, rather than its effect at one tuned configuration. We therefore chose the candidate values around the MinAtar DQN baseline. Young and Tian [2019] scaled DQN's replay capacity, target-update period, exploration schedule and replay warm-up down by a factor of ten relative to Mnih et al. [2015], giving 10^5, 1000, 10^5 and 5,000 steps. Each candidate range in Table I contains the corresponding baseline value, as do the batch size of 32 and the single hidden layer of the baseline. The three learning rates span one order of magnitude and are roughly centred, on a logarithmic scale, on the 2.5×10^{-4} used by Mnih et al. [2015] and by the MinAtar baseline, although that value is not itself one of them. Every run starts by filling the buffer with 5,000 transitions from uniformly random actions, the baseline's warm-up.

The learning-rate range was set from published values that were tuned for RMSProp with other constants (Section III-C), and we did not check before the trial that it contained the best rate for DQN under our recipe. The additional experiments show that it did not: in three of the four games, DQN did better at smaller rates (Section IV-F). The main trial's effects are therefore averages over a population in which most configurations use too large a learning rate for DQN under our recipe. This does not bias the paired comparisons within the population, but it limits the population to which they apply, and some effects change outside it.

Five arms were trained on every individual: DQN as the control and the four extensions as treatments. Within an individual, all arms share the configuration, the training seed and the evaluation seeds, so every comparison with DQN is paired. The extensions' own settings were fixed at published values rather than sampled: $\alpha = 0.6$ and $\beta_0 = 0.4$ for PER, the values Schaul et al. [2016] chose for its proportional variant; $n = 3$ for n-step DQN, as in Rainbow [Hessel et al. 2018]; and $K = 10$ for Averaged DQN, one of the values tested by Anschel et al. [2017]. PER's priority constant, $\kappa = 10^{-6}$, is our choice; it only has to be small enough not to change the priorities of transitions with non-zero error.

B. Environments

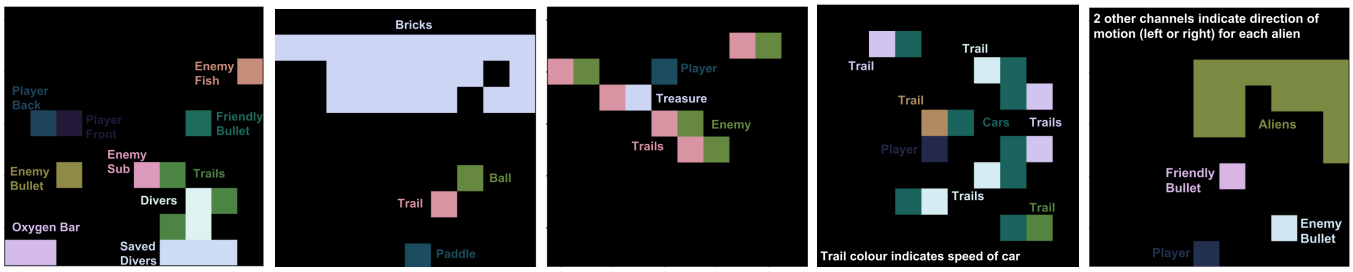
MinAtar [Young and Tian 2019] re-implements five Atari games on a 10×10 grid (Fig. 3). The agent observes a $10 \times 10 \times n$ binary array with one channel per kind of object: $n = 4$ in Breakout and Asterix, 7 in Freeway and 6 in Space Invaders. We chose MinAtar because it keeps the decision-making challenges of the Atari games while making each run cheap enough for a population of configurations; its authors designed it for exactly this kind of thorough, reproducible experiment [Young and Tian 2019]. We kept MinAtar's default settings, including sticky actions with probability 0.1, which repeat the previous action at random and make the games stochastic.

The main trial uses Breakout, Asterix, Freeway and Space Invaders, giving $4 \times 5 \times 30 = 600$ runs. Seaquest was also run but is reported separately, because our evaluation protocol initially failed on it: in MinAtar's Seaquest, oxygen only runs down while the submarine is below the surface, so a greedy agent that never dives is never killed and its episode never ends. The main trial aborted any run whose evaluation episode passed 100,000 steps. Seaquest jobs started for four of the five arms (the n-step DQN job failed at start-up), and this guard stopped all four. The additional experiments below repeat Seaquest with the evaluation episode capped instead.

C. Agent and Shared Training Recipe

Table II lists the training settings. All arms use the network of the MinAtar DQN baseline [Young and Tian 2019]: one convolutional layer of 16 filters followed by one or two hidden layers of 128 units. This is much smaller than the network of Mnih et al. [2015], which has three convolutional layers of 32, 64 and 64 filters and a hidden layer of 512 units. MinAtar has no frame skipping, so a frame is one environment step, and each run lasts 5×10^6 frames, as in the MinAtar baselines.

The recipe, meaning the optimiser and its constants, the loss, the update frequency and the exploration schedule, is shared by all arms and was fixed before training. It is not the recipe of DQN or of the MinAtar baseline, and Table III shows where it departs from them. Like both, we used RMSProp



Seaquest *Breakout* *Asterix* *Freeway* *Space Invaders*

Fig. 3. Visualisation of each MinAtar environment. Colour indicates the active channel at each spatial location; the representation provided by the environment consists of binary values for each channel, not RGB values. Reproduced from Young and Tian [2019].

TABLE II
TRAINING SETTINGS OF THE MAIN TRIAL

Setting	Value
Network	convolution (16 filters, 3×3, stride 1), then 1 or 2 hidden layers of 128 units (ReLU)
Optimiser, loss	RMSProp (decay 0.99, stabilising constant 10^{-8}, not centred, no momentum); unclipped squared TD error (4)
Discount, exploration	$\gamma = 0.99$; ε from 1.0 to 0.1, linear over the sampled duration
Updates	one gradient step per frame after 5,000 random transitions
Training length	5×10^6 frames (no frame skipping)
Evaluation	every 5×10^4 frames; 10 greedy episodes with seeds 1000–1009
PER	proportional; $\alpha = 0.6$; β from 0.4 to 1 over training; $\kappa = 10^{-6}$
n-step DQN	$n = 3$; no off-policy correction
Averaged DQN	$K = 10$ target networks, one stored at each target update

TABLE III
OUR SHARED RECIPE COMPARED WITH PUBLISHED DQN RECIPES

	DQN (Mnih)	MinAtar baseline	Ours (main trial)	Obando-Ceron and Castro
Optimiser	RMSProp	RMSProp	RMSProp	Adam
Decay	0.95	0.95	0.99	0.999
Constant	0.01	0.01	10^{-8}	3.125×10^{-4}
Centred	yes	yes	no	–
Loss	clipped	Huber	squared	squared
Gradient step every	4 steps	frame	frame	4 frames
Final exploration	0.1	0.1	0.1	0.01

Decay: of the running mean of squared gradients. Constant: added to the divisor of each step. DQN: Mnih et al. [2015], whose agent steps each span four frames and whose clipping of the error to $[-1, 1]$ is equivalent to the Huber loss. MinAtar baseline: the DQN script of Young and Tian [2019]. Obando-Ceron and Castro: their MinAtar DQN recipe [Obando-Ceron and Castro 2021], which our recipe experiment uses.

[Tieleman and Hinton 2012], but with PyTorch's default constants (second-moment decay 0.99, stabilising constant 10^{-8}, not centred) rather than theirs (0.95 and 0.01, centred). The difference may matter. RMSProp divides each gradient by the root mean square of that parameter's recent gradients plus the constant, so with a constant of 10^{-8} even a parameter whose gradients are tiny moves by about the full learning rate, whereas a constant of 0.01 damps such steps. At the same learning rate, our recipe may therefore take larger effective steps than the baseline, which would fit the finding that the smallest learning rates did best (Sections IV-D and IV-F).

We used the plain squared TD error of (4), without clipping. DQN clipped its error terms, which is equivalent to the Huber loss, and the MinAtar baseline uses the Huber loss. We chose the squared error because it is the loss in which DQN and every extension above are written, so that no arm's errors are clipped. Obando-Ceron and Castro [2021] found the squared loss much better than the Huber loss with Adam in every environment they tested, including MinAtar; in their comparison on 60 Atari games, however, the Huber loss tended to do better with RMSProp. One gradient step per frame, the same for every arm, matches the MinAtar baseline; DQN updated once every four agent steps. The discount 0.99 and final exploration rate 0.1 are those of both. Taken together, RMSProp with these constants, an unclipped squared loss and an update every frame make a recipe that is shared by all arms but not standard.

The recipe is not tuned for any arm, and that is deliberate. Because every arm uses the same recipe and the same configurations, any difference between arms is caused by the extension, so the comparisons are internally consistent. This answers the question faced by a practitioner who adds an extension to an existing DQN pipeline without re-tuning it. It does not answer which algorithm is best when each is properly tuned: an extension may need a different recipe from DQN, as PER needed a smaller step size in Schaul et al. [2016], and a recipe that suits one arm may handicap another. A negative result in the main trial therefore means that the extension did not help under this recipe and in this population, not that it cannot help. The additional experiments measure how much the conclusions depend on this choice.

The extensions were implemented as follows. PER stores priorities in a sum tree and draws each mini-batch by stratified sampling: the total priority is split into equal segments and one transition is drawn from each. n-step DQN stores, for each transition, the discounted sum of the next three rewards and the state three steps later; at the end of an episode, the shorter remaining sums are stored without a bootstrap term. Averaged DQN stores a copy of the target network at every target update and averages the last ten copies as in (10), or all

of them while fewer than ten exist.

D. Evaluation and Primary Outcome

Every 5×10^4 frames, training pauses and the current network plays 10 evaluation episodes greedily, that is with $\varepsilon = 0$, so that the evaluation measures what the agent has learnt rather than its exploration. The ten episodes use the same environment seeds (1000–1009) for every individual, arm and checkpoint, so the arms face exactly the same game situations. The mean return over the ten episodes, G_i at checkpoint i, is one point on the evaluation curve, giving 100 points per run.

The primary outcome is the area under this curve (AUC), computed with the trapezoidal rule:

$$\mathrm{AUC} = \sum_{i=1}^{99} \frac{G_i + G_{i+1}}{2}\,\Delta x, \qquad \Delta x = 5 \times 10^4 \text{ frames}. \tag{11}$$

Each term is the area of a strip between two neighbouring checkpoints: its width is the 5×10^4 frames between them and its height is the average of their returns. We report the AUC in units of 10^6 (return $\times$ frames). Dividing the AUC by the 4.95×10^6 frames it spans gives the average evaluation return over training, so a high AUC requires both learning fast and reaching a high level. That is why we use it as our measure of sample efficiency.

E. Statistical Analysis

The unit of analysis is the individual. The ten evaluation episodes at a checkpoint, and the 100 checkpoints of a run, all measure the same agent; treating them as independent observations would exaggerate the sample size. Each run therefore contributes one number, its AUC.

For each game and treatment, we form the 30 paired differences

$$d_j = \mathrm{AUC}_j^{\mathrm{treatment}} - \mathrm{AUC}_j^{\mathrm{DQN}}, \qquad j = 1, \ldots, 30, \tag{12}$$

one per individual. Pairing pays off whenever a configuration that suits DQN also suits the treatment. The variance of the mean difference is

$$\mathrm{Var}(\bar{d}) = \frac{\sigma_T^2 + \sigma_C^2 - 2\rho\,\sigma_T \sigma_C}{N}, \tag{13}$$

where σ_T and σ_C are the standard deviations of the treatment's and DQN's AUC across individuals and ρ is their correlation. When the configuration drives both arms in the same direction, ρ is large and positive, and the covariance term cancels much of the variance between configurations that an unpaired comparison would have to carry.

The primary test is the two-sided paired t-test. It asks whether the mean difference $\bar{d}$ is larger than chance would produce if the treatment had no effect, using the statistic

$$t = \frac{\bar{d}}{s_d / \sqrt{N}}, \tag{14}$$

where s_d is the standard deviation of the 30 differences, so the denominator is the standard error of $\bar{d}$. Under the null hypothesis of no mean effect, t follows a t-distribution with $N - 1 = 29$ degrees of freedom, and the p-value is the probability of a $|t|$ at least as large as the one observed.

We run 16 such tests, four treatments in four games, and each test has a chance of a false positive. If each test used the usual threshold of 0.05, the probability of at least one false positive among 16 independent tests would be $1 - 0.95^{16} \approx 0.56$. Because our conclusions compare effects across games, we treat all 16 tests as one family and apply the Bonferroni correction [Dunn 1961]: a test counts as significant only if $p \le \alpha = 0.05/16 = 0.003125$. The probability of at least one false positive is at most the sum of the 16 individual probabilities, $16\alpha = 0.05$, whether or not the tests are independent. The matching confidence interval for the mean difference is $\bar{d} \pm t_{1-\alpha/2,\,29}\, s_d/\sqrt{N}$, a 99.69% interval, which we divide by DQN's mean AUC to express it as a percentage.

We report each effect as a percentage of DQN's mean, $\Delta = 100\,\bar{d}/\overline{\mathrm{AUC}}^{\mathrm{DQN}}$, together with the number of individuals, out of 30, on which the treatment beat DQN. With no effect, about 15 wins would be expected.

As our one sensitivity check, we repeat every comparison with the Wilcoxon signed-rank test [Wilcoxon 1945]. It ranks the absolute differences $|d_j|$ from smallest to largest, sums the ranks of the positive differences, and asks whether that sum is unusually large or small. Because it uses ranks, a few extreme configurations cannot dominate it, which complements the t-test, whose mean can be pulled by outliers. The check is needed: the t-test assumes that the mean difference is roughly normally distributed, and the Shapiro–Wilk test [Shapiro and Wilk 1965] rejects normality of the differences for 6 of the 16 comparisons. We report both tests for every comparison, rather than choosing between them with a normality test, because such a pre-test changes the error rates of the test that follows [Rochon et al. 2012].

All other analyses are exploratory and are reported in the appendices: the ratio scale, Holm's procedure, secondary endpoints, the reliability of smaller evaluations and a regression on the sampled hyperparameters.

F. Additional Experiments

Under the shared recipe, the main trial found PER and n-step returns harmful, unlike earlier component studies, and found the smallest learning rate best for almost every arm (Section IV-D). We ran two additional experiments to help explain these findings, and a third that repeats Seaquest with a corrected evaluation. All of them use the same code, environments and evaluation as the main trial.

Recipe and positive control. If our PER and n-step DQN are correct, they should help in a setting where they are known to help. Obando-Ceron and Castro [2021] found multi-step returns to be the most useful addition to DQN on MinAtar. Their recipe differs from ours in three ways (Table III): the Adam optimiser [Kingma and Ba 2015] (stabilising constant 3.125×10^{-4}), one gradient step every four frames, and a final exploration rate of 0.01; like ours, it uses the squared loss. We trained DQN, PER and n-step DQN on Asterix and Freeway

in two settings. The first applies their recipe to the seven individuals of our population with learning rate 10^{-4}, keeping each individual's other hyperparameters. The second uses their published DQN configuration (learning rate 2.5×10^{-4}, replay capacity 10^5, batch 32, target period 1000, exploration decay over 250,000 frames, warm-up 1,000) with five seeds. Their paper does not state the number of hidden layers for MinAtar, so we used one, as in the MinAtar baseline. To check DQN itself, the second setting was also run for DQN in Breakout and Space Invaders.

Lower learning rates. Because the smallest learning rate in the population, 10^{-4}, was the best for almost every arm, we trained all five arms on the same seven individuals at 3×10^{-5} and 10^{-5}, under the shared recipe, in all four games. These seven are not a random sample of the population: they were selected because their learning rate was 10^{-4}, and only their learning rate was changed.

Seaquest with capped evaluation. We trained all five arms on all 30 individuals in Seaquest, with each evaluation episode stopped at 100,000 steps and scored with the reward collected so far.

These experiments were designed after the main results were known, and the first two use only five or seven configurations. We therefore treat them as exploratory: we report effects and win counts, without significance tests, and read their conclusions as suggestions rather than findings. Seaquest has all 30 individuals, so it receives the primary analysis as its own family of four tests, with threshold $0.05/4 = 0.0125$.

G. Implementation

The agents were implemented in PyTorch [Paszke et al. 2019] (version 2.13) with the Gymnasium [Towers et al. 2024] interface to MinAtar (version 1.0.15) and trained on CPUs on the cluster of the University of the Witwatersrand. Before every cluster job, unit tests and analytic checks verify each arm's targets and replay. Among them, PER's sampling probabilities and importance weights are compared with analytic values, n-step returns with an independent episode-level computation, and n-step DQN with $n = 1$ is checked to reproduce DQN's network weights exactly through the full training loop.

IV. Results

We first check that our DQN behaves like published DQN, then answer RQ1 with the main trial, turn to the additional experiments for RQ2, and end with RQ3. Each subsection opens with the question it answers. Numbers are in the tables and figures; the text states what they show.

A. Does Our DQN Match Published Results?

Table IV compares the final return of our DQN with the DQN baseline of Young and Tian [2019]. The comparison is approximate: their curve averages training episodes, which include exploration, while ours averages greedy evaluation episodes, and their numbers must be read from a figure. It can, however, reveal an implementation that is clearly too weak.

TABLE IV
Final Return of DQN Compared with the Published MinAtar DQN

	Published	Shared recipe			Published
Game	DQN	All 30	10^{-4} (7)	10^{-5} (7)	config (5)
Breakout	≈10	13.9	22.0	17.4	51.6
Asterix	≈17	3.8	7.6	30.8	34.0
Freeway	≈50	38.4	54.8	56.3	59.5
Seaquest	≈20	2.0	8.1	–	–
Space Invaders	≈45	35.9	65.7	97.6	164.7

Published DQN: approximate final return read from Fig. 3 of Young and Tian [2019] (mean of the last 100 training episodes, with exploration, at the best learning rate, 30 seeds). Our columns: mean greedy evaluation return over the last ten evaluations, averaged over all 30 individuals, over the seven individuals with learning rate 10^{-4}, over the same seven at 10^{-5}, and over five seeds of the published configuration of Obando-Ceron and Castro [2021]. Seaquest uses capped evaluation.

Averaged over the whole population, our DQN falls short of the published one in four of the five games. This is expected, because most of our configurations use a larger learning rate than suits DQN, as the following sections show.

At a learning rate of 10^{-5}, our DQN exceeds the published level in Breakout, Asterix, Freeway and Space Invaders, and with the published configuration of Obando-Ceron and Castro [2021] it exceeds it in all four. This gives no sign of a weak DQN implementation, and suggests that its low population average reflects the population's learning rates.

Seaquest is the exception: our DQN stays well below the published level even at the smallest learning rate tested there. Section IV-G shows why: under greedy evaluation, many agents learn to wait at the surface, which scores nothing.

B. RQ1: Which Extensions Change Sample Efficiency?

Table V gives the 16 comparisons. Nine of them are significant in the primary t-test, and the Wilcoxon test agrees on eight of the nine. Every effect in this subsection is relative to DQN under the shared recipe and in the sampled population.

Averaged DQN is the clearest success. It was significantly better than DQN in Asterix, Freeway and Space Invaders, it beat DQN on most individuals in each of these games, and both tests agree.

Double DQN helped little. Its only significant gain, in Space Invaders, is the weakest of the nine findings: the Wilcoxon test does not confirm it, and the lower end of its confidence interval is barely above zero.

Under this recipe, PER and n-step DQN were harmful. PER was significantly worse in Freeway and Space Invaders, and n-step DQN in Asterix, Freeway and Space Invaders. Both rarely beat DQN on any individual in these games.

No extension changed the result in Breakout. The confidence intervals there are narrow enough to rule out large effects in either direction.

C. How Do the Differences Arise During Training?

The learning curves in Fig. 4 show when the arms separate. In Breakout, all arms rise quickly and then overlap for the rest of training, which is why no AUC differs.

TABLE V
MAIN TRIAL: MEAN AUC (10^6, STANDARD DEVIATION IN PARENTHESES) AND PAIRED COMPARISONS WITH DQN ($n = 30$ PER ARM)

Game	Arm	AUC	Δ (%)	99.69% CI (%)	Wins	p_t	p_W
Breakout	DQN	65.2 (16.6)					
	Double DQN	65.3 (16.9)	$+0.1$	$[-6.2, +6.5]$	14	0.94	0.92
	PER	66.7 (18.3)	$+2.3$	$[-10.8, +15.3]$	17	0.58	0.36
	n-step DQN	68.1 (20.0)	$+4.5$	$[-8.4, +17.3]$	19	0.27	0.24
	Averaged DQN	65.9 (17.6)	$+1.1$	$[-8.6, +10.7]$	17	0.72	0.72
Asterix	DQN	16.8 (14.1)					
	Double DQN	21.5 (21.7)	$+28.2$	$[-15.9, +72.3]$	17	0.048	0.13
	PER	14.0 (9.6)	-16.5	$[-47.9, +14.8]$	14	0.10	0.27
	n-step DQN	3.3 (2.4)	-80.0	$[-131.3, -28.8]$	2	$\mathbf{2.3 \times 10^{-5}}$	$\mathbf{1.2 \times 10^{-6}}$
	Averaged DQN	26.4 (25.1)	$+57.5$	$[+3.9, +111.1]$	25	$\mathbf{0.0017}$	$\mathbf{4.2 \times 10^{-4}}$
Freeway	DQN	159.8 (58.9)					
	Double DQN	146.2 (64.8)	-8.5	$[-19.0, +1.9]$	10	0.014	0.019
	PER	103.4 (63.7)	-35.3	$[-55.5, -15.2]$	3	$\mathbf{4.1 \times 10^{-6}}$	$\mathbf{1.0 \times 10^{-7}}$
	n-step DQN	64.9 (54.9)	-59.4	$[-86.1, -32.7]$	0	$\mathbf{6.7 \times 10^{-8}}$	$\mathbf{1.9 \times 10^{-9}}$
	Averaged DQN	193.2 (52.4)	$+20.9$	$[+8.5, +33.3]$	24	$\mathbf{7.7 \times 10^{-6}}$	$\mathbf{1.1 \times 10^{-5}}$
Space Invaders	DQN	168.8 (90.3)					
	Double DQN	188.0 (100.6)	$+11.4$	$[+0.1, +22.6]$	21	$\mathbf{0.0029}$	0.012
	PER	84.4 (65.4)	-50.0	$[-68.0, -32.1]$	1	$\mathbf{7.1 \times 10^{-10}}$	$\mathbf{9.3 \times 10^{-9}}$
	n-step DQN	133.4 (106.0)	-21.0	$[-38.2, -3.8]$	6	$\mathbf{4.7 \times 10^{-4}}$	$\mathbf{1.9 \times 10^{-4}}$
	Averaged DQN	201.7 (111.0)	$+19.5$	$[+8.4, +30.6]$	26	$\mathbf{4.1 \times 10^{-6}}$	$\mathbf{3.8 \times 10^{-6}}$

Δ: mean paired difference as a percentage of the DQN mean, with its Bonferroni-adjusted 99.69% confidence interval. Wins: individuals, out of 30, on which the treatment's AUC exceeded DQN's. p_t: two-sided paired t-test (primary); p_W: Wilcoxon signed-rank test (sensitivity). Bold: $p \leq 0.05/16$.

In Asterix, n-step DQN barely improves at all, while the other arms improve steadily. Its deficit is a failure to learn, not slower learning.

In Freeway, Averaged DQN pulls ahead early and stays ahead, so its gain comes from both faster learning and a higher final level.

In Space Invaders, PER starts level with DQN but falls during the first million frames and stays well below DQN for the rest of training. PER therefore does not merely learn more slowly; its performance first gets worse.

D. How Much Do Configurations Matter?

Fig. 5 shows how widely the AUC varies between configurations. Within a single arm, the spread between configurations is often as large as the difference between arms. This is the situation in which pairing matters: an unpaired comparison of a few configurations could easily favour the wrong arm.

The learning rate explains much of this spread (Fig. 6). For every arm in every game except n-step DQN in Asterix and Freeway, the smallest learning rate, 10^{-4}, gave the highest mean AUC. A regression that adjusts for the other five hyperparameters confirms that the learning rate changes the AUC of DQN more than almost any extension does (Appendix C).

n-step DQN responds to the learning rate in the opposite way in Freeway. It failed at 10^{-4}, the rate at which every other arm did best, and partly recovered at the larger rates. A study that tuned each method separately would never see this interaction.

E. RQ2: Is the Harm of PER and n-Step Returns Due to the Recipe?

Table VI answers this question. With the published configuration of Obando-Ceron and Castro [2021], both PER and n-step DQN beat DQN on every seed in Asterix, and matched or slightly beat it in Freeway. This agrees with their finding that multi-step returns help DQN on MinAtar, and it indicates that our implementations of both extensions work.

Applying their recipe to our own configurations closed most of the gap. Under the shared recipe, n-step DQN lost to DQN on all fourteen game–configuration pairs and PER on thirteen of them. Under their recipe, PER beat DQN on six of the seven configurations in Asterix and came close to DQN in Freeway, and the mean deficit of n-step DQN shrank by at least two thirds in both games.

The recipe also raised every arm, including DQN, several-fold in Asterix. The shared recipe is therefore not only unfavourable to PER and n-step returns; it is a weaker recipe overall at these configurations.

n-step DQN still trailed DQN in Asterix on our configurations, while it led on the published configuration. Its remaining mean deficit in Asterix comes mostly from two of the seven configurations, on which its AUC fell below half of DQN's; on the other five it stayed close to DQN. The published configuration also differs from these seven in its learning rate, replay capacity, batch size, exploration decay and warm-up, and our experiments do not separate these factors.

F. RQ2: Do the Effects Survive a Smaller Learning Rate?

Fig. 7 and Table VII show what happens below 10^{-4}. A smaller learning rate made DQN much stronger in Asterix and Space Invaders and stronger in Freeway at the smallest rate,

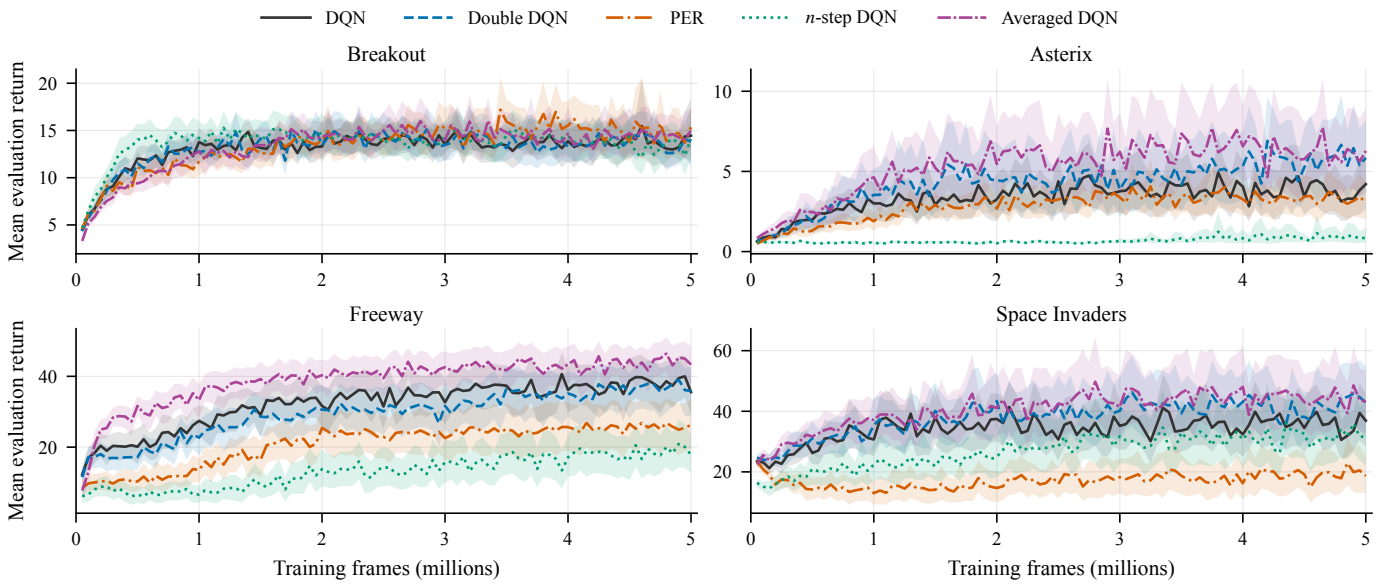


Fig. 4. Mean greedy evaluation return during training in the main trial ($n = 30$ individuals per arm). Shaded bands are pointwise 95% bootstrap intervals over individuals [Efron 1979]; the curves are not smoothed.

TABLE VI
DQN, PER AND n-STEP DQN UNDER THE SHARED RECIPE AND THE RECIPE OF OBANDO-CERON AND CASTRO [2021] (MEAN AUC, 10^6)

		Shared recipe, 7 configurations			Obando-Ceron–Castro recipe, same 7			Published configuration, 5 seeds		
Game	Arm	AUC	Δ (%)	Wins	AUC	Δ (%)	Wins	AUC	Δ (%)	Wins
Asterix	DQN	35.1			170.6			145.9		
	PER	23.1	−34.1	1/7	195.0	+14.3	6/7	187.3	+28.4	5/5
	n-step	3.0	−91.5	0/7	127.7	−25.1	0/7	175.7	+20.4	5/5
Freeway	DQN	205.4			280.3			281.5		
	PER	145.4	−29.2	0/7	266.2	−5.0	0/7	283.5	+0.7	4/5
	n-step	16.1	−92.2	0/7	256.4	−8.5	5/7	291.5	+3.6	5/5

Shared recipe: the main-trial runs of the seven individuals with learning rate 10^{-4}. Obando-Ceron–Castro recipe: the same seven individuals trained with Adam, one update every four frames and final exploration 0.01. Published configuration: their DQN hyperparameters, five seeds. Δ: mean paired difference from DQN as a percentage of DQN's mean. Wins: individuals or seeds on which the arm's AUC exceeded DQN's. Exploratory; no significance tests.

but not in Breakout. The trial's learning-rate range therefore stopped above the best rate for DQN in most games.

The advantage of Averaged DQN shrank at the smaller rates and was gone at the smallest. At 10^{-4} it beat DQN on all seven configurations in Asterix, Freeway and Space Invaders. At 3×10^{-5} it still beat DQN on all seven in Freeway, but on only two in Asterix and five in Space Invaders; at 10^{-5} it beat DQN on at most five of the seven in any game. Double DQN's gain in Asterix had already disappeared at 3×10^{-5}. These results rest on seven configurations selected for their learning rate.

PER stayed harmful at the smaller rates in Asterix and Freeway, where it lost to DQN on every configuration at both rates. In the other two games it recovered. In Space Invaders, PER's deficit shrank from -39.4% at 10^{-4} to -7.6% at 10^{-5}, and in Breakout, where PER was level with DQN at 10^{-4}, it was 29.9% better at 10^{-5} and beat DQN on six of the seven configurations.

n-step DQN improved relative to DQN as the learning rate fell, but not smoothly. In Freeway it stayed far behind at 3×10^{-5} and nearly caught up at 10^{-5}; in Breakout and Space Invaders it moved ahead of DQN at both smaller rates; in Asterix it stayed behind. In the main trial, other configurations with larger learning rates also did better in Freeway (Section IV-D). This suggests that its failure there depends on the combination of the learning rate with the other hyperparameters, not on the step size alone.

G. Does Averaged DQN Also Help in Seaquest?

With capped evaluation, Seaquest shows the same pattern as the main trial (Appendix D, Table XIII). Averaged DQN was better than DQN on most of the 30 configurations, both tests agree, and no other extension gave a result that both tests agree on. The scores are very low for every arm, because a greedy agent that has not learnt to dive and resurface, the exploration problem that Young and Tian [2019] identified in this game, can survive at the surface indefinitely without scoring.

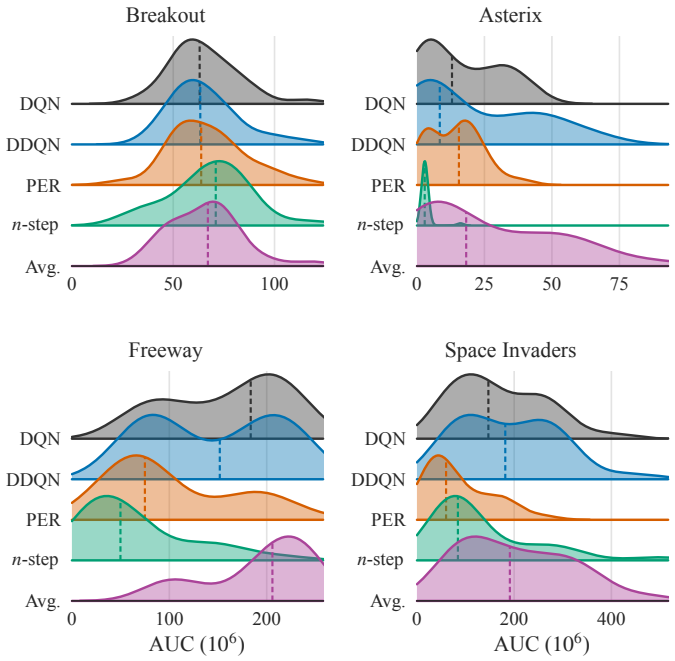


Fig. 5. Distribution of the AUC over the 30 individuals of each arm in the main trial, in the style of Pretorius et al. [2020]: Gaussian kernel density estimates (Scott's bandwidth, clipped at zero), each scaled to its own peak height. Dashed lines mark the medians. DDQN: Double DQN; Avg.: Averaged DQN.

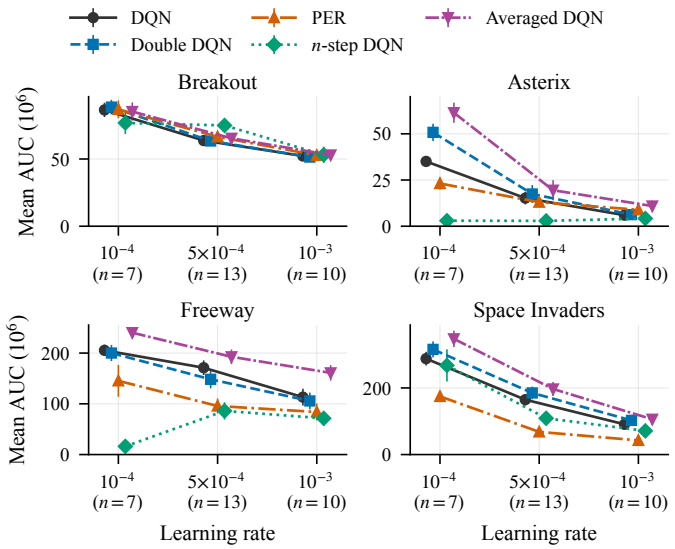


Fig. 6. Mean AUC by sampled learning rate in the main trial, with ±1 standard error over the n individuals at each rate. The subgroups also differ in their other hyperparameters, so the comparison is descriptive; Appendix C adjusts for them.

H. RQ3: Would Smaller Evaluations Have Agreed?

Appendix B repeats the main comparisons as smaller or more typical studies would have run them, and the answer is that they would often have reached weaker or different conclusions.

Single-checkpoint endpoints, the mean of the last ten evaluations or the final evaluation alone, agreed in direction with all nine findings but detected fewer of them. They have less power than the AUC, not different conclusions.

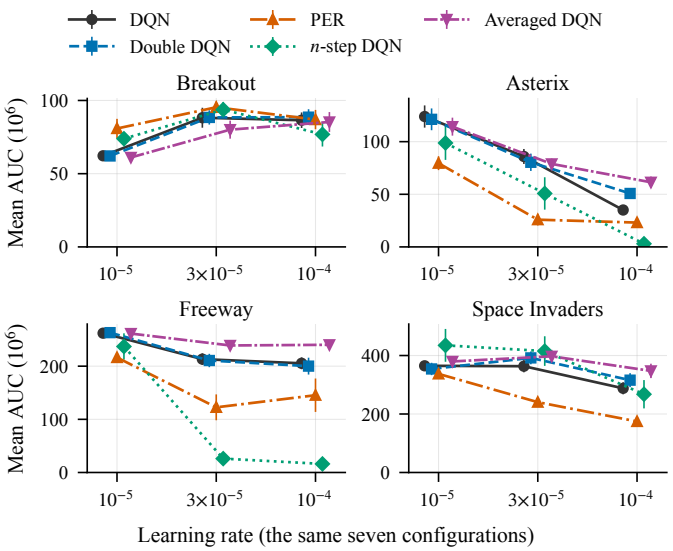


Fig. 7. Mean AUC of every arm on the seven individuals with learning rate 10^{-4} in the population, retrained at 3×10^{-5} and 10^{-5} under the shared recipe, with ±1 standard error.

TABLE VII
EFFECT OF EACH EXTENSION AT SMALLER LEARNING RATES (SAME SEVEN INDIVIDUALS)

Game	Arm	10^{-5}	3×10^{-5}	10^{-4}
Breakout	DQN	62.2	88.2	86.5
	DDQN	−0.2 (4)	+0.2 (4)	+2.4 (3)
	PER	+29.9 (6)	+8.1 (5)	+0.5 (3)
	n-step	+18.7 (7)	+6.3 (5)	−11.2 (2)
	Avg.	−1.8 (4)	−9.2 (1)	−1.4 (3)
Asterix	DQN	123.9	85.9	35.1
	DDQN	−2.2 (2)	−6.4 (2)	+44.8 (6)
	PER	−35.8 (0)	−69.9 (0)	−34.1 (1)
	n-step	−20.3 (0)	−40.9 (1)	−91.5 (0)
	Avg.	−7.8 (2)	−8.2 (2)	+75.2 (7)
Freeway	DQN	262.0	213.2	205.4
	DDQN	+0.4 (5)	−1.2 (3)	−2.6 (3)
	PER	−17.5 (0)	−42.5 (0)	−29.2 (0)
	n-step	−9.5 (3)	−87.8 (0)	−92.2 (0)
	Avg.	−0.3 (2)	+12.1 (7)	+16.9 (7)
Space Invaders	DQN	364.8	363.5	287.9
	DDQN	−2.9 (4)	+7.8 (5)	+9.7 (6)
	PER	−7.6 (2)	−33.8 (0)	−39.4 (0)
	n-step	+19.2 (5)	+14.2 (5)	−7.0 (3)
	Avg.	+4.1 (5)	+9.3 (5)	+20.7 (7)

DQN rows: mean AUC (10^6). Other rows: Δ (%) relative to DQN at the same learning rate, with the number of the seven individuals on which the arm beat DQN in parentheses. The 10^{-4} column is from the main trial. Exploratory; no significance tests.

Trials with a few configurations usually got the direction right but seldom reached significance. A small study that finds no effect therefore says little about whether an effect exists.

Comparing the best configuration of each arm with the best of DQN, as separately tuned studies effectively do, reversed two findings: it would have reported n-step DQN as better

than DQN in Space Invaders and PER as better in Freeway.

V. Related Work

This section describes the studies closest to ours and explains how our work differs from each.

Component studies. Hessel et al. [2018] combined six extensions into Rainbow and removed each in turn on 57 Atari games; removing prioritised replay or multi-step returns hurt most. Their ablation removes one component from a tuned combination, with one configuration. We instead add one component to DQN, over a random population of configurations; Section VI-C discusses what our results imply for theirs.

Obando-Ceron and Castro [2021] repeated Rainbow's additive and ablative studies on small environments, including MinAtar, with Adam and a learning rate swept for each component, and found multi-step returns to be the most useful addition to DQN on MinAtar. Their study varies the component and the learning rate with the rest fixed; ours varies six hyperparameters at random and pairs every comparison.

Fedus et al. [2020] showed that uncorrected n-step returns benefit from a larger replay capacity, whereas DQN does not. Their experiment varies the replay setting directly; in our population the replay capacity is one of six sampled factors, and our subgroups were too small to show its effect on n-step DQN.

Fujimoto et al. [2020] showed that a loss evaluated on prioritised samples has the same expected gradient as a different loss on uniform samples, so the effect of PER depends on the loss it is combined with. This is a theoretical result; our trial is an empirical test of PER with one loss and one optimiser, and Section VI uses the equivalence to interpret PER's deficit.

Anschel et al. [2017] introduced Averaged DQN and showed on Atari that it reduces the variance of the target and stabilises learning. They compared it with DQN on fixed configurations; we compare it over a population of configurations.

Evaluation methodology. Henderson et al. [2018] showed that hyperparameters, seeds, environments and codebases can each change reported results substantially. We take this as the motivation for our design: rather than fixing these factors, we sample the hyperparameters and pair every comparison on them.

Machado et al. [2018] revisited the evaluation protocols of the Arcade Learning Environment and introduced sticky actions. We follow their recommendation through MinAtar's sticky actions.

Colas et al. [2018] related the number of seeds to the error rates of statistical tests, and Agarwal et al. [2021] showed that point estimates over few runs can mislead and recommended interval estimates and the probability of improvement. Our win counts are a paired version of their probability of improvement, and our subsampling analysis measures directly how often small trials miss real effects.

Jordan et al. [2020] argued that tuning hyperparameters and then reporting performance answers the wrong question, and evaluated algorithms whose hyperparameters are drawn at random from a fixed distribution in every trial. Patterson et al. [2024] give a broader guide to empirical design in reinforcement learning. We share the idea of sampling hyperparameters from a distribution, but give every arm the same draws, which turns each comparison into a paired one.

Pretorius et al. [2020] brought the randomised control trial to deep learning, sampling network designs from a population and training identical copies under every intervention. We apply their design to reinforcement learning, where training is noisier and each run far more expensive, and to algorithmic extensions rather than network initialisation.

TABLE VIII
Strength of the Evidence for Each Conclusion

Conclusion	Evidence	Status
Under the shared recipe, Averaged DQN improves on DQN in Asterix, Freeway and Space Invaders	Main trial, $n = 30$, both tests	Established
Under the shared recipe, PER reduces the AUC in Freeway and Space Invaders, and n-step DQN in Asterix, Freeway and Space Invaders	Main trial, $n = 30$, both tests	Established
Double DQN improves on DQN in Space Invaders	Main trial, t-test only	Weak
No extension has a large effect in Breakout	Main trial, narrow intervals	Established
Our PER and n-step DQN can help DQN	Published configuration, 5 seeds, 2 games	Suggested
The shared recipe causes their deficit	7 configurations, 2 games; recipe components changed together	Suggested; cause not isolated
Averaged and Double DQN help mainly when the learning rate is too large	7 configurations selected by learning rate	Suggested
Small, single-checkpoint and best-configuration evaluations can miss or reverse effects	Resampling of the main trial	Established for these data

VI. Discussion

A. What the Evidence Supports

Our experiments differ in how much weight they can bear, and Table VIII separates them. The main trial, with 30 randomly sampled configurations, a pre-specified outcome and a correction for 16 tests, establishes how each extension changed sample efficiency under the shared recipe and in this population. It does not establish how the extensions compare when each is tuned, or under other recipes.

The additional experiments are exploratory (Section III-F). They show that the main trial's conclusions can change when the recipe or the learning rate changes, but they do not isolate why: the recipe experiment changes the optimiser, the update frequency and the final exploration rate together, and the learning-rate experiment uses seven configurations selected for their learning rate. The explanations below are therefore hypotheses that fit the data, not established causes.

B. RQ2: Why Do Our Results Differ from Published Ones?

The differences share a common thread: the effect of an extension appears to depend on how it interacts with the optimiser and the learning rate. The learning rate alone does not fix the size of an update. RMSProp divides each gradient by a running root mean square, so its constants change the effective step at a given rate (Section III-C), and Adam, which Rainbow used because it was less sensitive to the learning rate than RMSProp [Hessel et al. 2018], behaves differently again. The extensions in turn change the errors that drive the updates: PER replays the transitions with the largest errors more often, n-step targets sum several rewards and so vary more than one-step targets [Sutton and Barto 2018], and Averaged and Double DQN produce steadier or less optimistic targets. Whether an extension helps can therefore depend on whether the base updates are too large or too small. Our data are consistent with this in each case below.

PER and n-step returns are harmful in our main trial, unlike in the MinAtar study of Obando-Ceron and Castro [2021], and the positive control suggests that the recipe is responsible. With that study's recipe and configuration, both extensions help, as reported there (Section IV-E). Since both recipes use the squared loss, the difference lies in the optimiser and its constants, the update frequency or the final exploration rate (Table III). Our experiments change these together, so they cannot say which of them matters.

PER's deficit is not caused by the step size alone. Schaul et al. [2016] lowered the step size for PER because prioritised samples have larger errors and give larger gradients, and a smaller learning rate did help PER in Space Invaders and Breakout, but not in Asterix and Freeway (Section IV-F).

The equivalence of Fujimoto et al. [2020] suggests a reason that involves the loss. If the priorities were up to date, PER would draw transition i with probability proportional to its current $|\delta_i|^\alpha$ and multiply its loss by a weight proportional to $(|\mathcal{D}|P(i))^{-\beta}$, so the product would be proportional to $|\delta_i|^{\alpha(1-\beta)}$. With the squared loss, PER's expected gradient would then be the gradient of a uniformly sampled loss proportional to $|\delta|^{2+\alpha(1-\beta)}$. In practice the priorities are stale, because each is updated only when its transition is replayed, so this holds only approximately. With $\alpha = 0.6$ and $\beta_0 = 0.4$, our PER starts out behaving roughly like DQN with a loss of power 2.36, which weights large errors even more heavily than the squared loss does, and the power falls to two only as β reaches one at the end of training. Schaul et al. [2016] instead combined PER with clipped errors, for which the power for large errors lies between one and two, and Fujimoto et al. [2020] use this equivalence to argue that prioritisation suits the Huber loss better than the squared loss. Under our recipe, the update direction may thus be dominated by the few transitions with the largest errors. The loss cannot be the whole explanation, because the recipe of Obando-Ceron and Castro [2021] also uses the squared loss and PER helped there in Asterix; if the loss matters, it matters in combination with the optimiser or the update frequency.

n-step DQN's deficit depends on both the learning rate and the recipe: it shrank or reversed at smaller learning rates in three games and under the other recipe in both games tested (Sections IV-E and IV-F). Only in Asterix under the shared recipe did it fail at every learning rate, and with the published configuration it beat DQN there on every seed. That configuration has a replay capacity ten times that of six of the seven configurations we retrained, and Fedus et al. [2020] found that uncorrected n-step returns benefit from a larger replay capacity; but the configuration differs in other ways too, and the one retrained configuration with the larger capacity behaved like the rest, so we cannot attribute the difference to replay capacity.

Averaged DQN's gain agrees with Anschel et al. [2017], who presented it as a way to stabilise learning, and its disappearance at smaller learning rates (Section IV-F) suggests when this matters. One possible reason is that averaging ten target networks smooths the target, which compensates for updates that are too large; when the updates are already small, there is little left to smooth.

Double DQN helped less than in van Hasselt et al. [2016], whose Atari agents showed large overestimation. We did not record Q-value estimates, so we cannot tell whether overestimation was small in MinAtar or whether reducing it did not raise the AUC. Double DQN's effect also shrank at smaller learning rates, as Averaged DQN's did. In Space Invaders, the only game where its gain was significant, it went from $+9.7\%$ at 10^{-4} to $+7.8\%$ at 3×10^{-5} and -2.9% at 10^{-5}; its non-significant gain in Asterix disappeared already at 3×10^{-5}.

Together, these results suggest that no extension has a fixed effect; its effect depends on the optimiser and the learning rate it is combined with. Averaged DQN and Double DQN appear to help mainly when the updates are too large, so a population dominated by large learning rates flatters them. PER and n-step DQN appear to be hurt by the shared recipe, and the learning rate is only part of the reason, since lowering it alone restored neither of them in Asterix.

C. Implications for Component Studies

Hessel et al. [2018] found prioritised replay and multi-step returns to be the two most crucial components of Rainbow, in that removing either caused a large drop in median performance. Our main trial finds the same two components to be the most harmful additions to DQN. The two results need not conflict, because they were obtained under different recipes. Rainbow used Adam, chose a learning rate four times smaller than DQN's from a sweep, prioritised transitions by its distributional loss rather than by the TD error, and tuned n; its ablations then removed each component from this tuned combination. Our trial added each component to DQN under RMSProp, with mostly large learning rates and no tuning. When we moved towards a recipe like Rainbow's, the conclusion moved towards Rainbow's: with Adam and a published configuration, both extensions helped, and with Adam on our own configurations, or at smaller learning rates under RMSProp, they partly recovered. Rainbow's ranking is

therefore best read as a statement about components within its recipe, not as an intrinsic property of the components.

The same holds for the MinAtar study of Obando-Ceron and Castro [2021]. We reproduce their finding that multi-step returns help DQN under their recipe, and we find the opposite under ours. Their conclusion and ours are both correct for the recipes they were obtained under, and neither, on its own, tells us how the component behaves under a third.

The reverse can also happen. Averaged DQN is not part of Rainbow, yet under our recipe it was the only extension that helped in more than one game. A component study that evaluates a fixed set of components under one recipe can therefore overstate the components that suit its recipe and miss those that matter under others. This is the concern of Henderson et al. [2018] and Jordan et al. [2020], applied to the question of which components matter.

D. Practical Implications

Our results suggest six practices for researchers who design experiments or evaluate new reinforcement learning algorithms. Each follows from a specific result above.

1) *Treat the training recipe as part of the claim.* Report the optimiser and its constants, the loss and the update frequency in full, and test a claim that an extension helps under more than one recipe. In our trial, PER and n-step returns went from harmful to helpful with the recipe and configuration of an earlier study, and most of their deficit disappeared under that recipe on our own configurations (Section IV-E).
2) *Check that the hyperparameter range contains the baseline's best setting under the recipe actually used.* Ranges taken from studies with another optimiser can miss it, as ours did, and some effects changed outside our range (Section IV-F). A short pilot of the control arm over a wider range could reveal this cheaply.
3) *Run a positive control.* Before reporting that an extension does not help, reproduce a published setting in which it does. This separates a faulty implementation from a recipe effect (Section IV-E).
4) *Pair comparisons over a random population of configurations, and report win counts as well as means.* Pairing removes the configuration from each comparison, and the population makes the result about more than one setting. Comparing the best configuration of each method reversed two of our findings, and trials with a few configurations usually missed real effects (Section IV-H).
5) *Use a whole-curve outcome and check the evaluation protocol in every environment.* Single-checkpoint endpoints agreed with the AUC but detected fewer effects, and in Seaquest the handling of long evaluation episodes decided whether the game could be evaluated at all (Sections III-B and IV-G).
6) *Record the quantities each extension is meant to change.* We recorded only returns, so the explanations of Section VI-B remain hypotheses. Logging the mean predicted value of the greedy action next to the discounted return actually obtained, the variance of the target, and PER's TD errors and importance weights would have cost almost nothing and would have tested each mechanism directly.

VII. Limitations

Each configuration was trained with one seed, so within an individual the effects of the configuration and of the seed cannot be separated. Both are shared across arms, so the comparisons remain paired.

The main trial's conclusions are conditional on one non-standard recipe (Table III) and on a learning-rate range that lies above the best rate for DQN in most games (Section III-A). The additional experiments that probe this are small, use selected configurations and change several factors together (Section VI-A), and Seaquest was not run at smaller learning rates. We recorded returns but no internal quantities, so the mechanisms behind the effects remain untested.

The extensions' own settings were fixed rather than sampled. With more compute, the population could also randomise the optimiser, the loss, the update frequency, the number of steps n and PER's exponent α, so that the trial would cover the recipe as well as the configuration.

VIII. Conclusion

We compared four DQN extensions in a paired randomised control trial over 30 sampled configurations on MinAtar. Under one shared recipe, Averaged DQN improved on DQN in three of four games, Double DQN helped little, and PER and multi-step returns reduced performance. The central lesson, however, is not this ranking but how conditional it is. With the recipe of an earlier study, PER and multi-step returns helped as reported there; at smaller learning rates, DQN became much stronger in most games and most of Averaged DQN's advantage disappeared; and smaller or best-configuration evaluations would have missed or reversed some effects. The follow-up experiments are small and do not isolate the cause, but together they point to an interaction between the optimiser, the learning rate and the extension. Claims that an extension matters, including those of component studies such as Rainbow, should therefore be read as claims about the recipe and configurations under which they were made, and tested across recipes and populations of configurations, with paired comparisons, positive controls and records of the quantities each extension is meant to change.

Acknowledgement

I express my sincere gratitude to my supervisors, Dr. Steven James and Prof. Benjamin Rosman, for their invaluable guidance, patience, and continuous support throughout the preparation of this research report.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320, 2021.

Oron Anschel, Nir Baram, and Nahum Shimkin. Averaged-DQN: Variance reduction and stabilization for deep reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 176–185, 2017.

Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? Statistical power analysis in deep reinforcement learning experiments, 2018. arXiv:1806.08295.

Olive Jean Dunn. Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293):52–64, 1961.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

William Fedus, Prajit Ramachandran, Rishabh Agarwal, Yoshua Bengio, Hugo Larochelle, Mark Rowland, and Will Dabney. Revisiting fundamentals of experience replay. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3061–3071, 2020.

Scott Fujimoto, David Meger, and Doina Precup. An equivalence between loss functions and non-uniform sampling in experience replay. In *Advances in Neural Information Processing Systems*, volume 33, pages 14219–14230, 2020.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

J. Fernando Hernandez-Garcia and Richard S. Sutton. Understanding multi-step deep reinforcement learning: A systematic study of the DQN target, 2019. arXiv:1901.07510.

Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Scott M. Jordan, Yash Chandak, Daniel Cohen, Mengxue Zhang, and Philip S. Thomas. Evaluating the performance of reinforcement learning algorithms. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 4962–4973, 2020.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. arXiv:1412.6980.

Daniël Lakens. Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and ANOVAs. *Frontiers in Psychology*, 4:863, 2013.

Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.

Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

Johan S. Obando-Ceron and Pablo Samuel Castro. Revisiting Rainbow: Promoting more insightful and inclusive deep reinforcement learning research. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 1373–1383, 2021.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. *Journal of Machine Learning Research*, 25(318):1–63, 2024.

Arnu Pretorius, Elan van Biljon, Benjamin van Niekerk, Ryan Eloff, Matthew Reynard, Steve James, Benjamin Rosman, Herman Kamper, and Steve Kroon. If dropout limits trainable depth, does critical initialisation still matter? A large-scale statistical analysis on ReLU networks. *Pattern Recognition Letters*, 138:95–105, 2020.

Justine Rochon, Matthias Gondan, and Meinhard Kieser. To test or not to test: Preliminary assessment of normality when comparing two independent samples. *BMC Medical Research Methodology*, 12:81, 2012.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016. arXiv:1511.05952.

S. S. Shapiro and M. B. Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3–4):591–611, 1965.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, USA, second edition, 2018.

Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments, 2024. arXiv:2407.17032.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.

Christopher J. C. H. Watkins and Peter Dayan. Technical note: Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Kenny Young and Tian Tian. MinAtar: An Atari-inspired testbed for thorough and reproducible reinforcement learning experiments, 2019. arXiv:1903.03176.

Appendix A
Secondary Endpoints and the Ratio Scale

Table IX repeats the main comparisons on two single-checkpoint endpoints: the mean of the last ten evaluations (the final 5×10^5 frames) and the final evaluation alone. Both agree in direction with all nine significant findings, but fewer of them reach significance, and no comparison that was not significant on the AUC becomes significant.

A percentage of DQN's mean treats that mean as known exactly, and its interval can extend below -100% although no arm can be more than 100% worse; this happens for n-step DQN in Asterix. Table X and Fig. 8 therefore also analyse the per-individual log ratios $\log(\mathrm{AUC}_j^{\mathrm{treatment}}/\mathrm{AUC}_j^{\mathrm{DQN}})$. Their mean, transformed back, is the geometric-mean ratio, and p_r is a one-sample t-test of the log ratios. Seven of the nine findings remain significant on this scale. The two exceptions are gains: Averaged DQN in Asterix, where two configurations collapsed and dominate the log scale, and Double DQN in Space Invaders. Holm's step-down procedure [Holm 1979], which is at least as powerful as the Bonferroni correction, rejects the same nine hypotheses, and so does a Bonferroni correction within each game. The standardised effect $d_z = \bar{d}/s_d$ [Lakens 2013] is also given.

TABLE IX
SECONDARY ENDPOINTS: PAIRED COMPARISONS WITH DQN

Game	Arm	Last ten evaluations Δ (%)	Last ten evaluations p_t	Final evaluation Δ (%)	Final evaluation p_t
Breakout	DDQN	−0.8	0.87	−4.4	0.65
	PER	+7.0	0.40	+6.2	0.62
	n-step	−4.6	0.42	−2.5	0.79
	Avg.	+4.8	0.31	+3.5	0.69
Asterix	DDQN	+53.9	0.0071	+39.0	0.060
	PER	−14.0	0.20	−27.0	0.058
	n-step	−76.4	$\mathbf{6.3 \times 10^{-4}}$	−79.3	**0.0013**
	Avg.	+62.4	**0.0023**	+49.3	0.020
Freeway	DDQN	−3.4	0.58	+1.0	0.92
	PER	−32.7	0.0032	−26.1	0.029
	n-step	−50.6	$\mathbf{3.1 \times 10^{-5}}$	−49.9	$\mathbf{2.0 \times 10^{-4}}$
	Avg.	+14.8	0.012	+21.9	0.0092
Space Invaders	DDQN	+18.2	0.0036	+16.1	0.060
	PER	−46.8	$\mathbf{4.5 \times 10^{-8}}$	−49.4	$\mathbf{3.3 \times 10^{-5}}$
	n-step	−12.1	0.048	−14.7	0.11
	Avg.	+22.6	**0.0014**	+17.3	0.15

Δ and p_t as in Table V; bold: $p_t \leq 0.05/16$. DDQN: Double DQN; Avg.: Averaged DQN.

TABLE X
RATIO-SCALE ANALYSIS OF THE AUC

Game	Arm	Ratio (99.69% CI)	d_z	p_r
Breakout	DDQN	1.00 [0.93, 1.08]	+0.01	0.94
	PER	1.01 [0.90, 1.15]	+0.10	0.70
	n-step	1.03 [0.89, 1.18]	+0.20	0.58
	Avg.	1.01 [0.91, 1.12]	+0.07	0.82
Asterix	DDQN	1.04 [0.69, 1.58]	+0.38	0.76
	PER	0.98 [0.68, 1.41]	−0.31	0.83
	n-step	0.29 [0.15, 0.57]	−0.92	$\mathbf{1.8 \times 10^{-6}}$
	Avg.	1.38 [0.89, 2.12]	+0.63	0.024
Freeway	DDQN	0.89 [0.78, 1.02]	−0.48	0.0083
	PER	0.58 [0.41, 0.82]	−1.03	$\mathbf{2.1 \times 10^{-5}}$
	n-step	0.31 [0.17, 0.57]	−1.31	$\mathbf{8.3 \times 10^{-7}}$
	Avg.	1.25 [1.07, 1.47]	+0.99	$\mathbf{8.8 \times 10^{-5}}$
Space Invaders	DDQN	1.10 [0.97, 1.26]	+0.59	0.023
	PER	0.43 [0.30, 0.60]	−1.64	$\mathbf{1.2 \times 10^{-8}}$
	n-step	0.74 [0.61, 0.88]	−0.72	$\mathbf{9.3 \times 10^{-6}}$
	Avg.	1.18 [1.05, 1.33]	+1.03	$\mathbf{1.3 \times 10^{-4}}$

Ratio: geometric mean of the per-individual AUC ratio to DQN, with its 99.69% interval. d_z: mean difference divided by the standard deviation of the differences. p_r: one-sample t-test of the log ratios; bold: $p_r \leq 0.05/16$.

APPENDIX B
RELIABILITY OF SMALLER EVALUATIONS

Table XI gives three views of how smaller studies would have fared. The probability of improvement is the fraction of individuals on which the treatment beat DQN, with a 95% Wilson interval [Wilson 1927]; it estimates the chance that the treatment wins on a configuration drawn at random. The best-of-30 column compares the best AUC of each arm with the best AUC of DQN, which mimics separately tuned studies and is biased upwards. The detection columns repeat each significant comparison on every subset of three or five individuals and on 10^5 random subsets of ten, and record how often a paired t-test at 0.05 finds the effect in the full trial's direction.

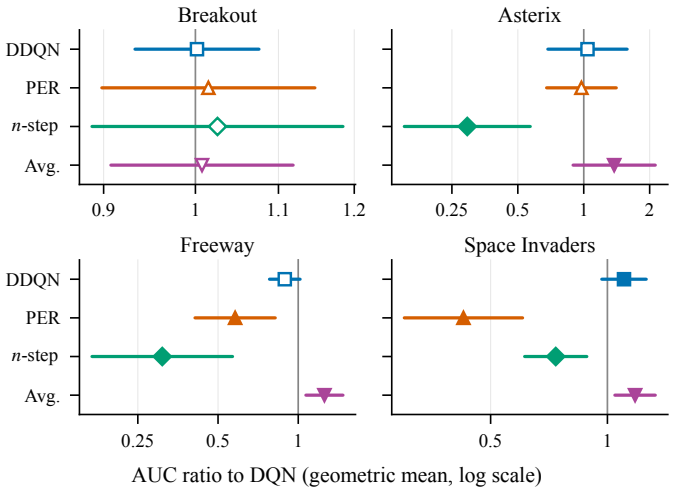


Fig. 8. Geometric-mean AUC ratio of each treatment to DQN, on a log scale, with 99.69% intervals. Filled markers: significant in the primary paired t-test. The horizontal scales differ between games.

TABLE XI
RELIABILITY OF SMALLER EVALUATIONS

Arm	PoI (95% CI)	Best of 30 Δ (%)	Detected (%) $k = 3$	5	10
Breakout					
DDQN	0.47 [0.30, 0.64]	−1.5	–	–	–
PER	0.57 [0.39, 0.73]	−2.5	–	–	–
n-step	0.63 [0.46, 0.78]	+3.3	–	–	–
Avg.	0.57 [0.39, 0.73]	+2.9	–	–	–
Asterix					
DDQN	0.57 [0.39, 0.73]	+50.8	–	–	–
PER	0.47 [0.30, 0.64]	−10.9	–	–	–
n-step	0.07 [0.02, 0.21]	−62.9	8	20	80
Avg.	0.83 [0.66, 0.93]	+106.1	7	14	42
Freeway					
DDQN	0.33 [0.19, 0.51]	−2.7	–	–	–
PER	0.10 [0.03, 0.26]	+2.0	13	33	98
n-step	0.00 [0.00, 0.11]	−8.4	14	54	100
Avg.	0.80 [0.63, 0.90]	+4.7	13	39	83
Space Invaders					
DDQN	0.70 [0.52, 0.83]	+13.0	6	9	37
PER	0.03 [0.01, 0.17]	−35.7	30	79	100
n-step	0.20 [0.10, 0.37]	+24.7	10	24	55
Avg.	0.87 [0.70, 0.95]	+18.4	17	42	86

PoI: probability of improvement with 95% Wilson interval. Best of 30: best AUC of the arm relative to the best of DQN. Detected: percentage of subsets of k individuals in which a paired t-test at $\alpha = 0.05$ is significant in the direction of the full trial, for the nine significant comparisons.

APPENDIX C
LEARNING-RATE REGRESSION

Table XII gives, for every arm and game, the ordinary least-squares regression of log AUC on the six sampled hyperparameters, each coded as a categorical factor with its first value as the reference (11 coefficients, 19 residual degrees of freedom). The learning-rate ratio is the back-transformed coefficient of 10^{-3} relative to 10^{-4}: the factor by which the

AUC changes when only the learning rate is raised. The partial R^2 is the share of the variance left by the other five factors that the learning rate explains. For DQN, the adjusted learning-rate effect is larger than the geometric-mean effect of the extension in 15 of the 16 comparisons; the exception is n-step DQN in Freeway, whose geometric-mean ratio of 0.31 (Table X) lies further below one than DQN's adjusted learning-rate ratio of 0.57.

TABLE XII
ADJUSTED LEARNING-RATE EFFECT ON LOG AUC

Game	Arm	LR ratio (95% CI)	Partial R^2	R^2
Breakout	DQN	0.56 [0.49, 0.63]	0.83	0.87
	DDQN	0.53 [0.48, 0.58]	0.92	0.94
	PER	0.60 [0.48, 0.74]	0.58	0.74
	n-step	0.55 [0.45, 0.67]	0.68	0.84
	Avg.	0.55 [0.48, 0.63]	0.83	0.88
Asterix	DQN	0.21 [0.11, 0.40]	0.58	0.82
	DDQN	0.11 [0.05, 0.22]	0.68	0.84
	PER	0.38 [0.18, 0.81]	0.28	0.65
	n-step	1.08 [0.74, 1.56]	0.06	0.37
	Avg.	0.18 [0.09, 0.37]	0.57	0.84
Freeway	DQN	0.57 [0.45, 0.73]	0.59	0.86
	DDQN	0.51 [0.42, 0.62]	0.74	0.93
	PER	1.10 [0.61, 2.01]	0.03	0.60
	n-step	4.12 [2.31, 7.34]	0.59	0.80
	Avg.	0.72 [0.56, 0.92]	0.30	0.72
Space Invaders	DQN	0.30 [0.26, 0.35]	0.93	0.96
	DDQN	0.32 [0.26, 0.39]	0.87	0.94
	PER	0.33 [0.20, 0.54]	0.55	0.83
	n-step	0.31 [0.23, 0.42]	0.78	0.89
	Avg.	0.27 [0.22, 0.32]	0.93	0.96

LR ratio: multiplicative change in AUC from a learning rate of 10^{-4} to 10^{-3}, adjusted for the other five hyperparameters, with 95% t-interval. Partial R^2: of the learning rate given the other factors. R^2: of the full model.

APPENDIX D
SEAQUEST WITH CAPPED EVALUATION

TABLE XIII
SEAQUEST WITH CAPPED EVALUATION (AUC, 10^6, $n = 30$ PER ARM)

Arm	AUC	Δ (%)	Wins	p_t	p_W	Capped
DQN	7.2 (17.8)					207
Double DQN	11.5 (23.2)	$+58.9$	15	0.019	0.38	340
PER	3.0 (3.1)	-58.8	11	0.15	0.14	237
n-step DQN	0.8 (0.6)	-89.3	7	0.052	$\mathbf{3.5 \times 10^{-5}}$	252
Averaged DQN	13.3 (20.5)	$+84.3$	26	**0.0066**	$\mathbf{9.9 \times 10^{-5}}$	77

Δ, wins, p_t and p_W as in Table V; bold: $p \leq 0.05/4$. Capped: mean number of evaluation episodes per run, out of 1,000, stopped at the 100,000-step cap.

Table XIII gives the Seaquest comparisons, tested as a family of four. Averaged DQN was better than DQN on most configurations, and both tests agree. No other extension gave a result that both tests agree on. Double DQN's mean is higher but it won on only half of the configurations, so its mean rests on a few runs. n-step DQN lost on most configurations, which the Wilcoxon test detects, but the t-test does not. The n-step AUCs are uniformly small; the large values are DQN's, on the few configurations on which DQN learnt something (its standard deviation is more than twice its mean), and these few large differences inflate the variance of the paired differences.

Across all arms and configurations, the Seaquest experiment has 15,000 evaluation points (150 runs of 100 checkpoints), and 3,786 of them contain at least one capped episode. Their mean return is 0.04, against 1.92 for evaluation points without a capped episode. A capped episode thus marks an agent that survives without scoring. Averaged DQN, the arm capped least often, also scores highest (Table XIII).